

PvrGPU Fragment Pipeline — Block Diagram, Data Path & Pseudocode

本文件以 block diagram、data path flow 及 pseudocode 形式詳述 [src/systemc/fragment/](#) 五個 SystemC module 的架構與執行行為。

這五個 module 負責處理 32×32 Tile 排程、ISP (Image Synthesis Processor) 頂點覆蓋測試與 HSR (Hidden Surface Removal) 隱藏面剔除、Fragment 2×2 Quad/4×2 Half-Stamp 生成、PBE (Pixel Back End) 色彩量化與 GLES Blending，以及 PbeWriteBack 影格緩衝區回寫。

0. Top-Level Architecture Block Diagram

Fragment Pipeline — Top-Level Block Diagram

5 個 SC_THREAD module 透過 bounded sc_fifo 串聯
FIFO 僅傳送 PipelineTxn handle；大量 payload 儲存在 MemoryPool

↓ sc_fifo<PipelineTxn> (來自 ParameterBuffer / FragmentDecoder)

① TileScheduler

IN: TileRecord[], TilePrimitiveRef[], ParameterTriangle[]
OUT: verified scheduled tile count
STAGE: kFragmentDecoded -> kTilesScheduled

↓ sc_fifo<PipelineTxn>

② ISP (Image Synthesis Processor / HSR)

IN: 32×32 TileRecords, ParameterTriangles, Edge Equations
OUT: FragmentCandidate[] (Visible/Rejected HSR owners)
STAGE: kTilesScheduled -> kVisibilityReady

↓ sc_fifo<PipelineTxn>

③ FragmentFrontend

IN: FragmentCandidate[], ParameterTriangles
OUT: FragmentInvocation[], FragmentQuad[] (2×2 quads),
FragmentShaderLane[] (4×2 half-stamps with helpers)
STAGE: kVisibilityReady -> kFragmentsReady

↓ sc_fifo<PipelineTxn>

[Shader Execution: PDS -> USC Slot -> USC Cluster]

[Texture Sampling: TextureUnit (SMP instructions)]
IN: FragmentQuad[], Varyings/Coefficients
OUT: FragmentOutput[] (PIXOUT0..3 raw float32 colors)
STAGE: kFragmentsReady -> ... -> kTextureComplete

↓ sc_fifo<PipelineTxn>

④ PBE (Pixel Back End)

IN: FragmentOutput[], FragmentInvocation[], BlendState
OUT: pbe_framebuffer (RGBA8 UNORM linear memory)
STAGE: kTextureComplete -> kPbeComplete

↓ sc_fifo<PipelineTxn>

⑤ PbeWriteBack

IN: pbe_framebuffer
OUT: DRAM Framebuffer write (0x10000000) & Readback
STAGE: kPbeComplete -> kFramebufferReady

1. Data Path Flow — 記憶體與 Payload 流向

以下展示各 module 在 MemoryPool 與 GPU DRAM 之間的讀取 (R) 與寫入 (W) 關聯：

Geometry & Decoder Output

W: tile_records[] (32×32 tile 邊界與 ref 範圍)
 W: tile_primitive_refs[] (依 API submit order 排序之 primitive 參照)
 W: parameter_triangles[] (24.8 fixed-point edge eq & depth & bbox)
 W: parameter_coefficients[] (1/W 及 Varying A/B/C 平面參數)

① TileScheduler

R: state.tile_records -> 載入 Tile 列表
 R: state.tile_primitive_refs -> 驗證 primitive 參照 (可為空)
 R: state.parameter_triangles -> 若 ref 為空則驗證 face-cull 不變量
 W: state.scheduled_tiles, state.counters.tiles_scheduled

② ISP (Image Synthesis Processor)

R: tile_records[], tile_primitive_refs[], parameter_triangles[]
 Process: 逐 tile 依序測試 1-sample center 之 Edge Equations
 Depth Test (DepthPass) 與深度插值 (InterpolateDepth)
 Opaque Early HSR (單一 owner 覆蓋) 或 Blending (保留全順序)
 W: state.fragment_candidates -> FragmentCandidate[] (Visible/Reject)
 W: state.active_fragment_invocations, covered_pixels, depth counters

③ FragmentFrontend

R: fragment_candidates[] -> 讀取 ISP 產生的可見片段
 R: parameter_triangles[] -> 讀取三角形邊界與頂點資料
 Process: 空間打包為 2×2 FragmentQuad (含 lane coverage mask)
 若含 Texture Sampling, 則打包為 4×2 SIMD half-stamp
 生成導數/紋理過濾所需的 Helper Pixels (非覆蓋區 lane)
 W: state.fragment_invocations -> FragmentInvocation[]
 W: state.fragment_quads -> FragmentQuad[]
 W: state.fragment_shader_lanes -> FragmentShaderLane[] (含 helper)
 W: state.counters.ps_invocations, fragment_groups

[USC Cluster & Texture Unit]

R: fragment_quads[], fragment_shader_lanes[], parameter_coefficients
 W: fragment_outputs[] -> FragmentOutput[] (PIXOUT0..3 float32)

④ PBE (Pixel Back End)

R: fragment_outputs[], fragment_invocations[], blend state
 Process: F32 轉 RGBA8 UNORM (四捨五入與 clamp)
 Clear Color 底色初始化
 Fixed-Function GLES Blending (Add/Sub/RevSub/Min/Max)
 Color Mask 寫入過濾 (RGBA 各通道獨立遮罩)
 W: state.pbe_framebuffer -> std::vector<uint8_t> (RGBA8 Linear)
 W: state.counters.pbe_pixels_written, pbe_fragment_writes

⑤ PbeWriteBack

R: state.pbe_framebuffer -> 取得完整的 RGBA8 影像資料
 Process: DRAM Write -> 寫入 0x10000000 實體位址
 DRAM Readback -> 讀回驗證資料一致性
 釋放 PBE 暫存 handle, 切換為 DRAM Framebuffer 狀態
 W: state.dram_framebuffer (Readback Handle)
 W: state.framebuffer_from_dram = 1, virtual_gpu_cycles 結算

2. TileScheduler 內部流程

TileScheduler — Internal Processing Flow

Step 1: 階段檢查與輸入載入

```
ASSERT state.stage == kFragmentDecoded
Load tile_records[] 及 tile_primitive_refs[]
ASSERT tile 數量大於 0
```

Step 2: Face-Cull 與空 Primitive 處理

```
IF primitive_refs 為空:
Load parameter_triangles[]
ASSERT state.raster_state.face_cull.enable == true
ASSERT 所有 parameter_triangles 皆為 non-rasterizable
ASSERT parameters.size() == c_primitives
```

Step 3: 計數與週期結算

```
state.scheduled_tiles = tiles.size()
state.counters.tiles_scheduled = tiles.size()
cycles = scheduler_base_cycles +
ceil(tile_count / scheduler_tiles_per_batch)
state.counters.tile_scheduler_cycles = cycles
state.stage = kTilesScheduled
WaitForCycles(cycles); StorePipelineState(); output
```

3. ISP (Image Synthesis Processor) 內部流程

ISP — Internal Processing Flow

Step 1: 狀態設定與 Early HSR 安全性判定

```
ASSERT state.stage == kTilesScheduled
opaque_early_hsr = (blend.enable == 0)
IF shader_may_discard OR !fragment_early_hsr_safe:
opaque_early_hsr = false (不可提早剔除)
early_depth_write = depth.write_enable &&
!shader_writes_depth
```

Step 2: 逐 Tile 逐 Primitive 走訪

```
FOR EACH TileRecord IN tiles:
FOR EACH TilePrimitiveRef IN tile:
ASSERT submit_ordinal 保持單調遞增 (API 順序)
triangle = parameters[ref.parameter_index]
計算 Tile 與 Triangle 邊界之重疊區間 [X, Y]
```

Step 3: 像素採樣點覆蓋測試 (CoversSample)

```
FOR y IN [y_begin, y_end), FOR x IN [x_begin, x_end):
sample_pos = (x * 256 + 128, y * 256 + 128)
測試 3 條 Edge Equations:  $a \cdot s_x + b \cdot s_y + c \geq 0$ 
(若為 0 則依 top-left inclusive 規則判定)
IF NOT covered: CONTINUE
```

Step 4: 深度測試 (Depth Test) 與插值

```
depth_val = InterpolateDepth(triangle, edge_values)
IF depth.test_enable:
    passes = DepthPass(compare_op, depth_val, stored_z)
IF NOT passes: 記錄 candidate.visibility=Rejected
```

Step 5: Early HSR 擁有權仲裁 (Owner Resolution)

```
IF passes:
    candidate.visibility = kVisible
IF opaque_early_hsr:
    IF 該像素已有舊 owner:
        candidates[previous_owner].visibility = Reject
        owner[pixel] = current_candidate_index
IF early_depth_write:
    depth_buffer[pixel] = depth_val
```

Step 6: 統計與輸出

```
visible_count = count(c.visibility == kVisible)
state.fragment_candidates = store(candidates)
state.active_fragment_invocations = visible_count
state.counters.hsr_rejected_fragments = total - vis
state.stage = kVisibilityReady; WaitForCycles()
```

4. FragmentFrontend 內部流程 (Quad & Stamp 構建)

FragmentFrontend — Internal Processing Flow

Step 1: 讀取可見片段與順序驗證

```
ASSERT state.stage == kVisibilityReady
candidates[] = LoadArray(state.fragment_candidates)
檢查 Opaque 模式下每像素僅有單一 Visible 擁有者
檢查 Blended 模式下多片段維持 submit_ordinal 順序
```

Step 2: 基礎 2×2 FragmentQuad 構建

```
quad_x = candidate.x / 2, quad_y = candidate.y / 2
quad_lane = (y % 2) * 2 + (x % 2)
生成 FragmentInvocation 並填入對應 FragmentQuad:
quad.coverage_mask |= (1 << quad_lane)
quad.write_mask |= (1 << quad_lane)
quad.invocation_indices[quad_lane] = index
```

Step 3: 紋理採樣 4×2 SIMD Half-Stamp 處理 (若啟用)

```
IF UsesTextureSampling(state):
    找出被 touched 的 4×2 half-stamps:
    half_stamp_id = (y / 2) * stamps_x + (x / 4)
    將每個 4×2 stamp 拆分為 2 個 2×2 quad
    FOR lane IN 0..3:
        IF 該 lane 位於 ISP 可見清單:
            quad.coverage_mask |= (1 << lane)
            shader_lane.helper = 0
        ELSE (位於邊界外但屬於同一 SIMD quad):
            quad.helper_mask |= (1 << lane) (輔助計算導數)
            shader_lane.helper = 1 (不進行實際色彩寫入)
    state.fragment_shader_lanes = store(shader_lanes)
```

Step 4: 輸出與管線推進

```
state.fragment_invocations = store(invocations)
state.fragment_quads = store(fragment_quads)
state.counters.ps_invocations = invocations.size()
state.fragment_groups = fragment_quads.size()
state.stage = kFragmentsReady; WaitForCycles()
```

5. PBE (Pixel Back End) 色彩合成與混合流程

PBE — Internal Processing Flow

Step 1: 初始化 Framebuffer 底色

```
ASSERT state.stage == kTextureComplete
Load fragment_invocations[] 及 fragment_outputs[]
建立 width * height * 4 位元組陣列
預設填入 state.raster_state.clear_color (RGBA8 UNORM)
```

Step 2: 逐 Fragment 讀取與 F32->UNORM8 轉換

```
FOR EACH FragmentOutput IN outputs:
ASSERT output 與 invocation 身份一致 (API 順序)
source[0..3] = FloatBitsToUnorm8(output.pixel_out)
(利用四捨五入與 [0.0, 1.0] 範圍鉗位轉換)
```

Step 3: Fixed-Function Blending 運算 (若啟用)

```
IF blend.enable:
dst_color[0..3] = framebuffer[pixel_offset..+3]
FOR c IN 0..3:
sf = FactorToUnorm8(source_factor, src, dst, c)
df = FactorToUnorm8(dest_factor, src, dst, c)
blended = BlendEquationUnorm8(equation, s, d, sf, df)
// Add: (s*sf + d*df + 127)/255
// Sub: (s*sf - d*df + 127)/255
// RevSub: (d*df - s*sf + 127)/255
// Min / Max: min(s, d) / max(s, d)
ELSE:
blended = source[c]
```

Step 4: Color Mask 寫入與狀態更新

```
IF color_mask & (1 << c):
framebuffer[pixel_offset + c] = blended
state.pbe_framebuffer = store(framebuffer)
state.counters.pbe_fragment_writes = outputs.size()
state.stage = kPbeComplete; WaitForCycles()
```

6. PbeWriteBack 記憶體回寫流程

PbeWriteBack — Internal Processing Flow

Step 1: 檢查 PBE Framebuffer 資料

```
ASSERT state.stage == kPbeComplete
驗證 pbe_framebuffer 大小 == width * height * 4
state.framebuffer_gpu_address = 0x10000000
```

Step 2: DRAM 實體寫入與讀回驗證 (Memory Mode)

```
IF memory_system != nullptr:  
memory_>Write(0x10000000, framebuffer.data())  
readback = memory_>Readback(0x10000000)  
ASSERT readback.data == framebuffer.data  
readback_handle = store(readback.data)  
state.dram_framebuffer = readback_handle  
state.framebuffer_from_dram = 1  
release(state.pbe_framebuffer)  
計算 virtual_gpu_cycles 並更新計數器  
state.stage = kFramebufferReady; completion.write()
```

Step 3: Transaction 模式 (若無直連記憶體)

```
ELSE:  
發送 MemoryTxn(kFramebuffer, kWrite) 至 SLC/DRAM
```

7. 關鍵資料結構與 Handle Map

7.1 Fragment 階段 MemoryPool Handles

Handle 名稱	儲存資料型別	生產者	主要消費者
tile_records	TileRecord[]	Tiler	TileScheduler, ISP
tile_primitive_refs	TilePrimitiveRef[]	Tiler	TileScheduler, ISP
parameter_triangles	ParameterTriangle[]	ParameterBuffer	ISP, FragmentFrontend
fragment_candidates	FragmentCandidate[]	ISP	FragmentFrontend
fragment_invocations	FragmentInvocation[]	FragmentFrontend	PDS, USC, PBE
fragment_quads	FragmentQuad[]	FragmentFrontend	PDS Engine
fragment_shader_lanes	FragmentShaderLane[]	FragmentFrontend	USC Cluster, TextureUnit
fragment_outputs	FragmentOutput[]	USC Cluster / TextureUnit	PBE
pbe_framebuffer	uint8_t[] (RGBA8)	PBE	PbeWriteBack
dram_framebuffer	uint8_t[] (RGBA8)	PbeWriteBack (DRAM Readback)	JsonReporter

7.2 FragmentCandidate 結構 (ISP 輸出)

FragmentCandidate

x, y : 螢幕像素座標 (uint32)
primitive_id : API 原始 Primitive 索引
parameter_index : ParameterTriangle 陣列索引
submit_ordinal : API 提交順序序號
sample_mask : 採樣遮罩 (預設 1)
depth : 插值後的深度值 (float32)
barycentric[3] : 重心座標 a, b, c (float32)
visibility : kVisible 或 kRejected (HSR)

7.3 FragmentQuad 結構 (2x2 USC 排程單位)

FragmentQuad

parameter_index : 所屬三角形索引
quad_id : 螢幕 2X2 空間網格 ID
submit_ordinal : 提交順序
coverage_mask : 4-bit 有效覆蓋遮罩 (bits 0..3)
helper_mask : 4-bit 輔助導數遮罩 (Texture)
write_mask : 4-bit 寫入允許遮罩
invocation_indices[4] : 對應 4 個 lane 之索引

8. Reference uArch 計時參數

各 Fragment 模組依據 **kReferenceUarch** 之設定進行事件驅動週期延遲：

參數名稱	模組	說明
scheduler_base_cycles	TileScheduler	排程啟動基礎週期
scheduler_tiles_per_batch	TileScheduler	每批次排程處理之 Tile 數
isp_base_cycles	ISP	ISP 運算基礎週期
isp_candidates_per_batch	ISP	每批次執行覆蓋測試之候選片段數
fragment_frontend_base_cycles	FragmentFrontend	前端分組基礎週期
fragment_lanes_per_batch	FragmentFrontend	每批次組裝 Quad 之 Lane 數
pbe_base_cycles	PBE	像素後端基礎處理週期
pbe_pixels_per_batch	PBE	每批次寫入 Framebuffer 之像素數
pbe_blend_fragments_per_batch	PBE	每批次執行 Blending 之片段數

9. TileScheduler Pseudocode

```
FUNCTION TileScheduler::Run():
    LOOP forever:
        txn = input_fifo.read()
        state = load(pool, txn.handle)
        ASSERT state.stage == kFragmentDecoded

        tiles[] = load(pool, state.tile_records)
        primitive_refs[] = load(pool, state.tile_primitive_refs)
        ASSERT tiles.size() > 0

        // 處理完全被剔除 (Face-Culled) 的狀況
        IF primitive_refs.empty():
            parameters[] = load(pool, state.parameter_triangles)
            ASSERT state.raster_state.face_cull.enable
            ASSERT all_of(parameters, [](p) { return p.rasterizable == 0; })
            ASSERT parameters.size() == state.counters.c_primitives

        state.scheduled_tiles = tiles.size()
        state.counters.tiles_scheduled = tiles.size()

        cycles = scheduler_base_cycles + ceil(tiles.size() / scheduler_tiles_per_batch)
        state.counters.tile_scheduler_cycles = cycles
        state.counters.renderer_cycles += cycles
        state.stage = kTilesScheduled

        wait(cycles)
        store(pool, txn.handle, state)
        output_fifo.write(txn)
```

10. ISP Pseudocode

```
FUNCTION ISP::Run():
    LOOP forever:
        txn = input_fifo.read()
        state = load(pool, txn.handle)
        ASSERT state.stage == kTilesScheduled

        opaque_early_hsr = (state.raster_state.blend.enable == 0)
        IF state.raster_state.shader_may_discard OR !state.fragment_early_hsr_safe:
            opaque_early_hsr = false
        early_depth_write = state.raster_state.depth.write_enable && !state.raster_state.shader_writes_depth

        tiles[] = load(pool, state.tile_records)
        primitive_refs[] = load(pool, state.tile_primitive_refs)
        parameters[] = load(pool, state.parameter_triangles)

        candidates = []
        owner_map = array(size=pixel_count, init=NO_OWNER)
        depth_buffer = array(size=pixel_count, init=clear_depth)

        FOR EACH tile IN tiles:
            FOR offset = 0 .. tile.primitive_ref_count - 1:
                ref = primitive_refs[tile.first_ref + offset]
                triangle = parameters[ref.parameter_index]

                x_range = intersect([tile.x0, tile.x1], [triangle.min_x, triangle.max_x])
                y_range = intersect([tile.y0, tile.y1], [triangle.min_y, triangle.max_y])

                FOR y IN y_range:
                    FOR x IN x_range:
                        sample_x = x * 256 + 128
                        sample_y = y * 256 + 128

                        // 1. 覆蓋測試 (CoversSample)
                        IF !CoversSample(triangle, sample_x, sample_y, edge_values):
                            CONTINUE

                        cand.x = x; cand.y = y
                        cand.primitive_id = triangle.key.api_primitive_id
                        cand.parameter_index = ref.parameter_index
                        cand.submit_ordinal = ref.submit_ordinal
                        cand.depth = InterpolateDepth(triangle, edge_values, cand.barycentric)

                        // 2. 深度測試 (Depth Test)
                        passes = true
                        IF depth.test_enable:
                            passes = DepthPass(depth.compare_op, cand.depth, depth_buffer[pixel_idx])

                        cand_idx = candidates.size()
                        candidates.append(cand)
                        IF !passes: CONTINUE

                        // 3. Early HSR 與 Depth Write
                        cand.visibility = kVisible
                        IF opaque_early_hsr:
                            IF owner_map[pixel_idx] != NO_OWNER:
                                candidates[owner_map[pixel_idx]].visibility = kRejected
                                owner_map[pixel_idx] = cand_idx

                        IF depth.test_enable AND early_depth_write:
                            depth_buffer[pixel_idx] = cand.depth

        state.fragment_candidates = store(pool, candidates)
        state.active_fragment_invocations = count_if(candidates, is_visible)
        state.stage = kVisibilityReady
        wait(isp_cycles)
        store(pool, txn.handle, state)
        output_fifo.write(txn)
```

11. FragmentFrontend Pseudocode

```
FUNCTION FragmentFrontend::Run():
    LOOP forever:
        txn = input_fifo.read()
        state = load(pool, txn.handle)
        ASSERT state.stage == kVisibilityReady

        candidates[] = load(pool, state.fragment_candidates)
        parameters[] = load(pool, state.parameter_triangles)

        invocations = []
        quad_map = map()
        fragment_quads = []

        // 1. 建立可見片段的 2x2 Quads
        FOR EACH cand IN candidates WHERE cand.visibility == kVisible:
            quad_x = cand.x / 2
            quad_y = cand.y / 2
            quad_id = quad_y * quads_x + quad_x
            quad_lane = (cand.y % 2) * 2 + (cand.x % 2)

            inv = MakeInvocation(cand, quad_id, quad_lane)
            inv_idx = invocations.size()
            invocations.append(inv)

            quad = quad_map.get_or_create(cand.parameter_index, quad_id)
            quad.coverage_mask |= (1 << quad_lane)
            quad.write_mask |= (1 << quad_lane)
            quad.invocation_indices[quad_lane] = inv_idx

        // 2. 紋理採樣特殊處理 (4x2 Half-Stamps 與 Helper Pixels)
        IF UsesTextureSampling(state):
            shader_lanes = []
            fragment_quads = []
            touched_stamps = find_touched_4x2_stamps(candidates)

            FOR EACH stamp IN touched_stamps:
                FOR child_quad IN {left_2x2, right_2x2}:
                    quad = MakeQuad(stamp, child_quad)
                    FOR lane IN 0..3:
                        lane_pos = quad_origin + offset(lane)
                        IF is_visible_in_isp(lane_pos):
                            quad.coverage_mask |= (1 << lane)
                            shader_lanes.append(MakeShaderLane(inv, helper=0))
                        ELSE:
                            quad.helper_mask |= (1 << lane)
                            shader_lanes.append(MakeShaderLane(inv, helper=1))
                    fragment_quads.append(quad)
            state.fragment_shader_lanes = store(pool, shader_lanes)

        state.fragment_invocations = store(pool, invocations)
        state.fragment_quads = store(pool, fragment_quads)
        state.stage = kFragmentsReady
        wait(frontend_cycles)
        store(pool, txn.handle, state)
        output_fifo.write(txn)
```

12. PBE Pseudocode

```
FUNCTION PBE::Run():
    LOOP forever:
        txn = input_fifo.read()
        state = load(pool, txn.handle)
        ASSERT state.stage == kTextureComplete

        invocations[] = load(pool, state.fragment_invocations)
```

```

outputs[] = load(pool, state.fragment_outputs)

// 1. 初始化底色
framebuffer = array(size=width * height * 4)
FOR pixel = 0 .. pixel_count - 1:
    framebuffer[pixel*4 .. +3] = FloatToUnorm8(state.raster_state.clear_color)

// 2. 逐 Fragment 色彩轉換與合成
FOR i = 0 .. outputs.size() - 1:
    inv = invocations[i]
    out = outputs[i]
    pixel_idx = out.y * state.width + out.x
    byte_offset = pixel_idx * 4

    source_color = FloatBitsToUnorm8(out.pixel_output[0..3])

    IF state.raster_state.blend.enable:
        dest_color = framebuffer[byte_offset .. +3]
        FOR c IN 0..3:
            sf = FactorToUnorm8(blend.src_factor, source_color, dest_color, c)
            df = FactorToUnorm8(blend.dst_factor, source_color, dest_color, c)
            blended = BlendEquationUnorm8(blend.equation, source_color[c], dest_color[c], sf, df)
            IF color_mask & (1 << c):
                framebuffer[byte_offset + c] = blended
        ELSE:
            FOR c IN 0..3:
                IF color_mask & (1 << c):
                    framebuffer[byte_offset + c] = source_color[c]

state.pbe_framebuffer = store(pool, framebuffer)
state.stage = kPbeComplete
wait(pbe_cycles)
store(pool, txn.handle, state)
output_fifo.write(txn)

```

13. PbeWriteBack Pseudocode

```

FUNCTION PbeWriteBack::Run():
    LOOP forever:
        txn = input_fifo.read()
        state = load(pool, txn.handle)
        ASSERT state.stage == kPbeComplete

        framebuffer_data = pool.read(state.pbe_framebuffer)
        expected_bytes = state.width * state.height * 4
        ASSERT framebuffer_data.size() == expected_bytes

        state.framebuffer_gpu_address = 0x10000000
        state.stage = kPixelDataMasterComplete
        wait(kPbeWriteBackLatency)

        // DRAM 實體寫入與 Readback 驗證
        IF memory_system != nullptr:
            memory.Write(0x10000000, framebuffer_data)
            readback = memory.Readback(0x10000000, expected_bytes)
            ASSERT readback.data == framebuffer_data

            state.dram_framebuffer = store(pool, readback.data)
            state.framebuffer_from_dram = 1
            pool.release(state.pbe_framebuffer)
            state.pbe_framebuffer = NULL

            state.stage = kFramebufferReady
            wait(memory_cycles)
            store(pool, txn.handle, state)
            completion_port.write(txn)
        ELSE:
            output_port.write(MemoryTxn(kFramebuffer, kWrite, 0x10000000))

```